

Multi-Modal Agentic AI System for Medical Diagnosis and Triage

Project Pipeline Workflow - Victor Oluwadare - Code Review Manager

Team GitHub Repository: <https://github.com/abhishek7112000/Week-3-Team-9>

Project Overview

This project constructs an autonomous, multi-modal AI agent that diagnoses chest and cardiovascular conditions by sequentially analysing patient-submitted symptoms, chest X-ray images, and structured clinical records. Three machine learning models are built, evaluated, and compared before being integrated into a LangGraph.js agent deployed as a Next.js web application.

Datasets and model assignments:

Model	Type	Dataset	Output
Model 1 — ChestVision	Custom Neural Network	NIH ChestX-ray14	14-label X-ray classification (ONNX)
Model 2 — ClinicalFusion	Hybrid (MLP + frozen OpenMed)	Synthea Synthetic EHR	Diagnosis category + severity score (ONNX)
Model 3 — OpenMed QA	Pre-trained + transfer learning	Medical QA Dataset (HF)	Professional patient-facing response (HF Inference)
Agent LLM — MedGemma	Instruction-tuned (no fine-tuning)	N/A — used as-is	Clinical synthesis and DiagnosisReport JSON (HF Inference)
RAG — Encyclopedia	Retrieval-augmented generation	A-Z Medical Encyclopedia	Double-check layer; ChromaDB knowledge base

1. Data Pre-Processing and Feature Engineering

Dataset 1 — NIH ChestX-ray14 (Model 1)

112,120 frontal chest X-ray images labelled with 14 thoracic conditions. The official NIH train/test patient-level split is used to prevent data leakage.

- Resize images to 224×224 (ResNet-50) or centre-crop from 256×256 (ViT-B/16); replicate single greyscale channel to 3-channel RGB; apply ImageNet normalisation (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]).
- Training augmentation: random horizontal flip (p=0.5), ±10° rotation, brightness/contrast jitter, random erasing (p=0.1). No vertical flip (anatomically invalid).
- Class imbalance: compute per-label positive weight (N^- / N^+) for BCEWithLogitsLoss; supplement with Asymmetric Loss (ASL, gamma_neg=4) for long-tail multi-label handling.

Dataset 2 — Synthea Synthetic EHR (Model 2)

Synthetic patient records filtered to cardiovascular and respiratory ICD-10 codes (I21, I25, I50, I48, J18, J44, J45). Feature engineering produces two parallel inputs for the hybrid model.

- Tabular branch input: standardised demographics, pivoted lab values (troponin, BNP, creatinine, Hgb, CRP), vital signs (latest and 90-day mean), binary medication flags, comorbidity flags, encounter counts, and derived terms (pulse pressure, shock index).
- Textual branch input: template-generated clinical summary per patient (e.g. "58-year-old male, prior MI, elevated troponin, on beta-blocker") tokenised for the frozen OpenMed encoder. Missing values: group-median imputation with binary indicator columns.
- Labels: 5-class diagnosis category (Cardiac / Respiratory / Overlap / Hypertensive Crisis / Other) and ordinal severity score 0–3 derived from troponin elevation, SpO2 < 94%, SBP > 180, and hospitalisation history.

Dataset 3 — Medical QA Dataset (Model 3)

- Load Malikeh1375/medical-question-answering-datasets from HuggingFace Hub. Tokenise Q→A pairs with the OpenMed tokeniser (max_length=512). Retain pairs relevant to chest and cardiovascular conditions for domain alignment.

RAG Knowledge Base — A-Z Medical Encyclopedia

- Chunk encyclopedia into 512-token segments (64-token overlap) using LangChain RecursiveCharacterTextSplitter. Embed each chunk with the OpenMed encoder CLS token (768-dim) and store in ChromaDB (HNSW index). Used by the agent’s RAG node at runtime only.

2. Model Training and Improvement

Model 1 — ChestVision (Custom Neural Network from Scratch)

Two architectures are built from scratch and compared; the best-performing is selected for deployment.

	ResNet-50	ViT-B/16
Architecture	50-layer residual CNN; GlobalAvgPool → Dropout(0.5) → Linear(2048,14) → Sigmoid	12 transformer blocks, 16×16 patches; LayerNorm → Linear(768,14) → Sigmoid
Loss	Asymmetric Loss ASL (gamma_neg=4, gamma_pos=0)	Asymmetric Loss ASL (same config, fair comparison)
Optimiser / LR	AdamW; 1e-3 cosine decay to 1e-5	AdamW; 1e-4 with 10-epoch linear warmup, cosine decay
Batch / Epochs	64 / 100 (early stopping patience=10)	32 / 100 (early stopping patience=10)
ONNX export	~98 MB (fits Vercel 250 MB limit without quantisation)	~330 MB raw; INT8 dynamic quantisation to ~85 MB

- Hyperparameter tuning: Optuna (50 trials, Bayesian), objective = macro mean AUC across 14 classes. Search space: lr [1e-4, 5e-3], dropout [0.3, 0.6], ASL gamma_neg [2, 6], weight decay [1e-5, 1e-3]. MedianPruner halts unpromising trials at epoch 20.

Model 2 — ClinicalFusion (Hybrid Model)

Two branches process the same patient in parallel; their outputs are concatenated and passed through a fusion classification network.

- Tabular MLP branch: Linear(input, 256) → BatchNorm → ReLU → Dropout(0.3) → Linear(256,128) → ReLU → Dropout(0.3) → Linear(128,64). All weights trained from scratch.
- Frozen OpenMed branch: patient text summary passed through OpenMed encoder with all weights frozen (requires_grad=False) during initial training. Outputs 768-dim CLS token. Optional: unfreeze top 2 layers after epoch 15 at lr=1e-6.
- Fusion: concatenate [64-dim tabular || 768-dim text] → Linear(832,256) → BatchNorm → ReLU → Dropout(0.4) → Linear(256,128). Two output heads: diagnosis (Softmax, 5 classes) and severity (Softmax, 4 classes). Combined loss: 0.6×CE(diagnosis) + 0.4×CE(severity).
- Training: AdamW lr=3e-4, batch=128, epochs=60 (early stop on weighted F1). Optuna tunes hidden dims, dropout, loss weight split. Ablation compares MLP-only, text-only, and full fusion.

Model 3 — OpenMed QA (Pre-Trained + Transfer Learning)

- Base model: OpenMed generative or extractive QA variant (MaziyarPanahi / OpenMed on HuggingFace). Fine-tuned on the Medical QA dataset using HuggingFace Trainer. Lower 60% of encoder layers frozen to preserve general biomedical knowledge.
- Optimiser: AdamW lr=2e-5, warmup_ratio=0.06, batch=16 (gradient accumulation ×4), epochs=5. After fine-tuning, model is pushed to HuggingFace Hub and served via the Inference Endpoints API.

3. Model Evaluation and Metrics

Model	Primary Metrics	Targets / Thresholds
Model 1ChestVision	Mean AUC-ROC (14-class macro average)Per-class AUC-ROC, Average PrecisionF1 at optimal threshold (Youden’s J)Grad-CAM localisation AUC (bounding boxes)ONNX CPU latency	Mean AUC ≥ 0.82 (CheXNet DenseNet-121 benchmark: 0.841)≥ 0.75 per class≥ 0.60 for top 5 conditionsReport vs annotated regions< 50 ms per image
Model 2ClinicalFusion	Macro F1 and weighted F1 (diagnosis)ROC-AUC one-vs-rest per classCohen’s Kappa (severity)Severity MAEFalse low rate (safety metric)ONNX CPU latency	Macro F1 ≥ 0.80; weighted F1 ≥ 0.85≥ 0.90 per class≥ 0.75< 0.40< 2% (critical patients misclassified as low severity)< 20 ms per record

Model 3OpenMed QA	ROUGE-L, BERTScoreExact Match / F1 token overlapTransfer gain vs base modelClinical tone review (manual)	ROUGE-L ≥ 0.70 ; BERTScore $\geq$ 0.85F1 token ≥ 0.80 Fine-tuned must exceed base by ≥ 5 F1 ptsPass team review (2+ members)
-------------------------	--	--

All three models are evaluated on held-out test sets with no overlap with training or validation data. A cross-model comparison table records primary metric, ONNX export size, and CPU inference latency for each variant. The ablation study for Model 2 (MLP-only vs text-only vs full hybrid) quantifies the contribution of each branch. Grad-CAM heatmaps for Model 1 and the confusion matrix for Model 2 are included in the final report to provide visual evidence of model behaviour.

4. Application Deployment

The application is built entirely in TypeScript. Python is used only for model training; no Python services run in production. ONNX models (Models 1 and 2) are bundled directly into Vercel serverless functions and executed via **onnxruntime-node**. Model 3 (OpenMed QA) and MedGemma-27B-IT are served via HuggingFace Inference Endpoints and called using **@huggingface/inference**.

Agent workflow: The LangGraph agent runs as a stateful directed graph. The patient enters symptoms and optionally uploads a chest X-ray via the AgentChat UI. The **intake_node** checks NeonDB for an existing clinical record (requesting upload if absent). The **route_node** invokes **xray_node** (Model 1 ONNX) and **clinical_node** (Model 2 ONNX) in parallel. Results are passed to the **synthesise_node**, which calls MedGemma-27B-IT to produce a structured DiagnosisReport. If confidence < 0.60, the **clarify_node** generates follow-up questions. The **rag_node** then queries ChromaDB to double-check findings against the medical encyclopedia. Finally, the **response_node** passes the report and RAG context to the fine-tuned OpenMed QA model to generate the patient-facing response. Triage level (EMERGENCY / URGENT / ROUTINE / SELF_CARE) determines whether **triage_node** books a Google Calendar appointment and sends Gmail confirmations.

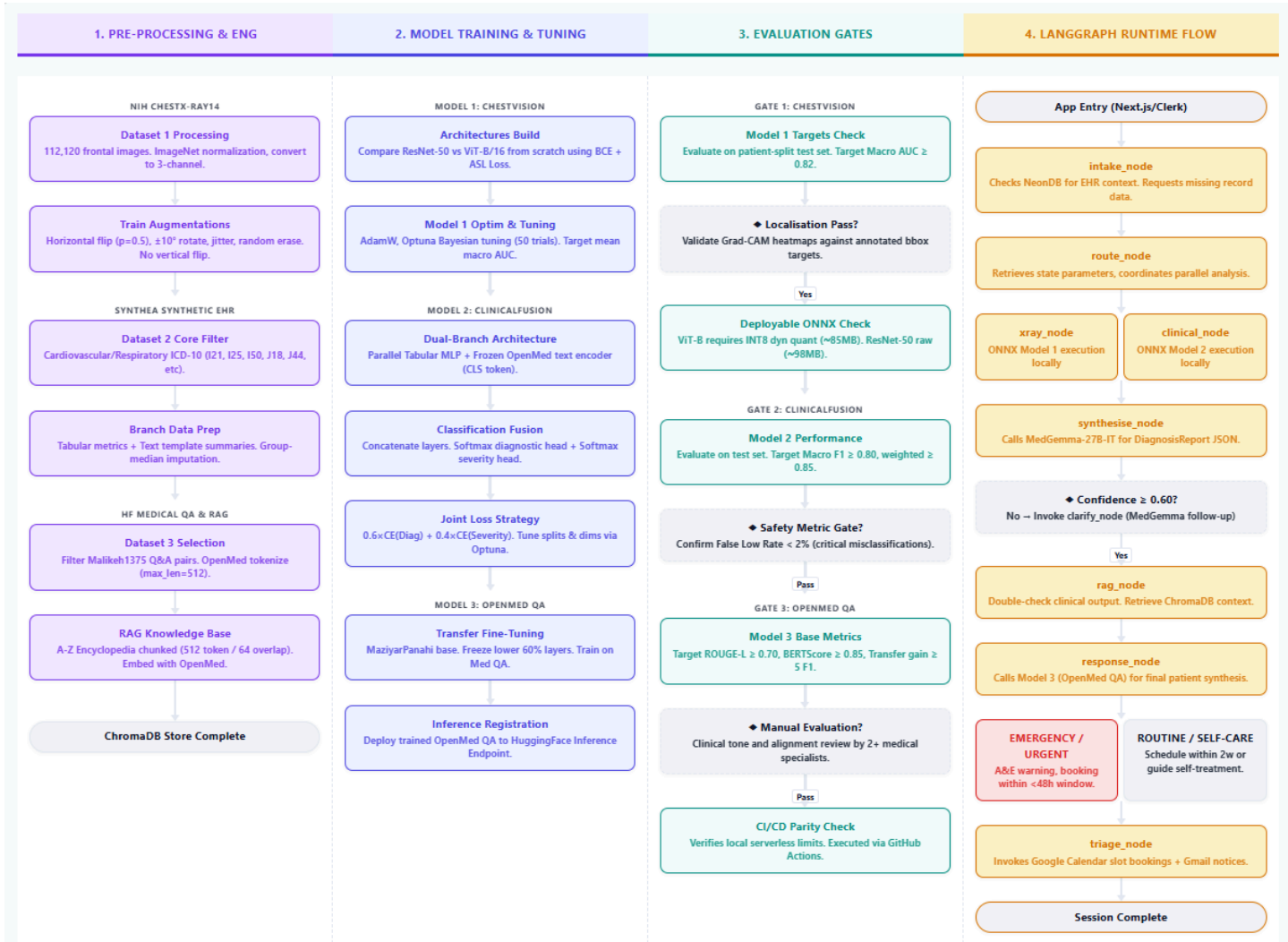


Figure 1: Pipeline Workflow